ECE 526L Spring 2025 Lab 4

Design of Eight Bit Register with Mutiplexer and D Flip Flop



Muhammad Umar
CSUN ID: 204533244

Instructor: Prof. Orod Haghighiara

I hereby attest that this lab report is entirely my own work. I have not copied either code or text from anyone, nor have I allowed or will I allow anyone to copy my work.

Name (printed): **Muhammad Umar**

Name (signed):  Date: 03/07/2025

INTRODUCTION

In this lab, we designed and implemented an **8-bit register** using instances of **2x1 multiplexers (MUX2_1)** and **D flip-flops (DFF)**. The primary objective of this lab was to understand the functionality of digital registers, the role of multiplexers in data selection, and the behavior of flip-flops in storing data.

In this design, each bit of the register is controlled by a **2x1 multiplexer**, which selects between the current stored value and the new input data based on an enable signal (**ena**). The selected data is then stored in a **D flip-flop**, which updates its output on the rising edge of the clock signal (**clk**). An active-low reset signal (**rst_n**) is also added to initialize the register outputs to zero.

To achieve this, we instantiated **eight 2x1 multiplexers** and **eight D flip-flops**, one for each bit of the register. The multiplexers and flip-flops were interconnected to ensure proper data flow and storage

This report documents the design process, including the Verilog implementation of the multiplexer, D flip-flop, and 8-bit register modules. It also includes the testbench used for simulation and the results obtained. The report concludes with an analysis of the design's performance, including the calculation of the maximum operating frequency and the behavior of the circuit when the clock frequency exceeds this limit.

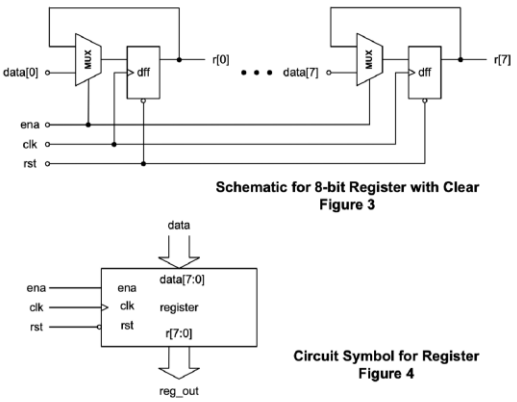


Figure 4.1 Eight bit Register Schematic

we will verify the functionality of an 8-bit register by checking its behavior under different conditions. We will test whether the register retains its previous value when **ena** is low, correctly updates when ena is high, and resets to zero when rst_n is asserted. Additionally, we will examine its response to timing violations by simulating a clock period shorter than the critical path delay.

METHODOLOGY AND TEST PLAN

1. **Multiplexer (MUX2_1.v):**
   - The 2x1 multiplexer is used to select between two input signals based on a control signal which is enable (ena) signal. In our design, it plays an important role in determining whether the register should retain its previous value or update with new data. We have introduced delays in the multiplexer like in previous lab to model realistic propagation delays, ensuring accurate simulation of real-world hardware behavior. The two **AND** gates operate in parallel, so

only one of them contributes to the critical path delay. Therefore, the total multiplexer delay is **11 ns**.

2. D Flip-Flop (dff.v):

The D flip-flop (DFF) is the fundamental storage element in our design. It captures and holds data on the rising edge of the clock signal. The output of the multiplexer is fed into the DFF, allowing the register to either retain or update its stored value depending on the enable (**ena**) signal.

3. 8-bit Register (register8bit.v):

In the register8bit module, we have instantiated **eight** D flip-flops to create an 8-bit storage unit. Each flip-flop stores a single bit of data. The multiplexer is used to determine whether each flip-flop should hold its previous value (when ena = 0) or update with new data (when ena = 1). The design ensures proper synchronization with the clock and allows for controlled data updates based on enable and reset signals which is active low.

4. Calculation of Total Critical Path Delay:

Each AND gate has a delay of 2.5ns. Since the two AND gates operate in parallel, the delay contribution is 2.5 ns.

Not gate has a delay of 1.5ns and or gate has a delay of 7ns. Flip-Flop Setup Time is 2ns. And Clock-to-Q Delay is 1ns.The total critical path delay is **14ns**.

5. Maximum Operating Frequency:

The maximum operating frequency can be determined by **1/critical time,** after calculating the result is **71.43Mhz** while the clock we are using is of **100MHz**.

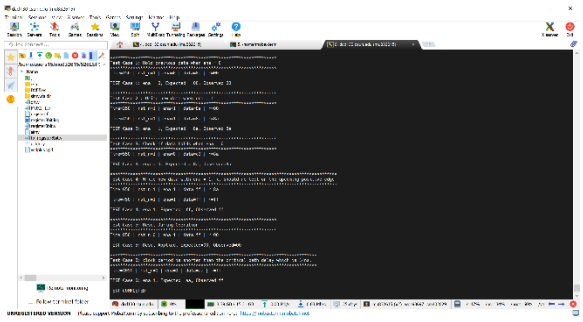


Figure 4.2 Terminal screenshot of the eight bit register simulation result.

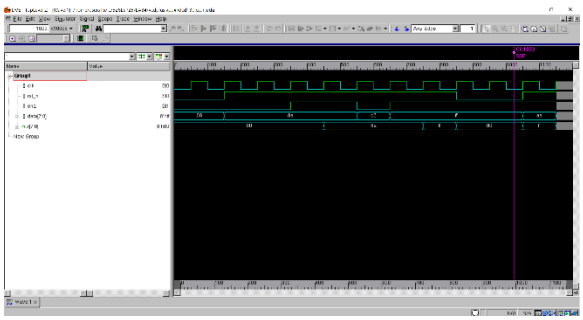


Figure 4.3 Waveview of Eight bit register

To verify the functionality of **8-bit register**, we analyze the simulation waveforms for various scenarios, making sure that the design behaves as expected under different conditions. The testbench systematically applies inputs and monitors the output (**r**) to validate each test case.

1. Hold Data (ena = 0)

- Initially, ena is set to **0**, meaning the register should retain its previous value.
- The input data is changed, but since ena = 0, the **MUX** selects the stored value from the flip-flops instead of the new data.

- We observe in the waveform that despite changing data, the register (r) does not update, confirming the correct behavior.

2. Write Data (ena = 1)

- ena is set to **1**, enabling the register to store new data.
- A new value is applied to data, the D flip-flops capture and store this value.
- The waveform shows that **r** updates to match data exactly after the critical delay passed which is **14ns** and we got the sample data on positive edge of clock, validating that the register correctly writes new data.

3. Data Hold Again (ena = 0)

- After successfully writing data, ena is set back to **0**, and data is changed again.
- Since ena = 0, the **MUX** continues to pass the stored value, preventing any updates.
- The waveform confirms that r does not change despite new input data, ensuring that the register holds its previous value correctly.

4. Write New Data (ena = 1)

- ena is re-enabled (**1**) and a different data value is provided.
- On the **next rising clock edge**, the register updates to reflect the new input.
- The waveform confirms that the output (r) changes only after the clock transition, ensuring correct synchronization and data-latching behavior.

5. Reset Operation

- The active-low **reset signal (rst_n)** is **asserted (0)** during normal operation, forcing all flip-flops to reset.
- After two clock cycle, rst_n is de-asserted (set to **1**), allowing normal operation to resume.
- The waveform shows that r is cleared to **0000_0000** immediately after reset, verifying that the reset mechanism works correctly.

6. Timing Violation Test (Clock Period < 14 ns)

- To check the register's behavior under **timing violations**, we reduce the clock period to **less than 14 ns**, making it shorter than the **critical path delay** of the design.
- If the clock period is too short, setup and hold time violations may occur, leading to unpredictable register output.
- By analyzing the waveform, we check for metastability, incorrect data storage, or glitches that may arise due to insufficient timing margins.

ANALYSIS OF RESULTS

The simulation results confirm the correct functionality of the **8-bit register**, verifying data retention, updates, and reset behavior. When ena = 0, the register correctly holds its previous value despite input changes, while setting ena = 1 allows new data to be stored on the next clock edge. The reset operation successfully forces the register to **0x00**, demonstrating proper asynchronous reset functionality. Additionally, a timing

violation test with a clock period below **14 ns** reveals potential setup and hold violations, emphasizing the importance of adhering to the maximum operating frequency of **71.43 MHz** for reliable performance.

CONCLUSION

Through this lab, I have gained a deep understanding of register design and its behavior under different conditions. By simulating various test cases, I was able to verify data retention, controlled updates, and reset functionality, reinforcing my knowledge of sequential logic and timing constraints. The inclusion of **MUX and DFF components** provided insight into how data is managed within the register, while the timing violation test emphasized the importance of operating within the critical path limitations. This hands-on experience has enhanced my grasp of digital design, simulation analysis, and the significance of clock synchronization in sequential circuits. All the design codes, testbench and log files are in appendices.

APPENDIX

Appendix A: MUX2_1.v

```
/*********************************************************************
***                                                                ***
*** EE 526 L Experiment #4            Muhammad Umar, Spring, 2025    ***
***                                                                ***
*** Experiment #4 Design 8bit register with instances of Mux and DFF ***
***                                                  ***
*** Filename: MUX2_1.v            Created by Muhammad Umar, 03/07/2025 ***
***                                                                ***
*** This module implements a logic circuit of Multiplexer 2x1 wit    ***
*** required delays.                                               ***

`timescale 1 ns / 100 ps

`define PRIMARY_OUT     5   // ns (primary outputs)
`define FAN_OUT_1       0.5 // ns (one output fanout)
`define TIME_DELAY_1    1   // ns (one input gates)
`define TIME_DELAY_2    2   // ns (two input gates)


module MUX2_1(OUT, A, B, SEL);
   // Port declarations
   output OUT;
   input A, B, SEL;

   // Internal variable declarations
   wire A1, B1, SEL_N;

   // The netlist with delays
   not  #(`TIME_DELAY_1 + `FAN_OUT_1) NOT1(SEL_N, SEL);       // NOT gate delay
   and  #(`TIME_DELAY_2 + `FAN_OUT_1)  AND1(A1, A, SEL_N);     // AND gate with 2
inputs
   and  #(`TIME_DELAY_2 + `FAN_OUT_1)  AND2(B1, B, SEL);      // AND gate with 2
inputs
   or   #(`TIME_DELAY_2 + `PRIMARY_OUT)  OR1(OUT, A1, B1);      // OR gate with 2
input and final output

endmodule
```

Appendix B: dff.v

```
/*************************************************************************
***                                                                    ***
*** EE 526 L Experiment #4            Muhammad Umar, Spring, 2025      ***
***                                                                    ***
*** Experiment #4 Design 8bit register with instances of Mux and DFF   ***
***                                                                    ***
*** Filename: dff.v               Created by Muhammad Umar, 03/07/2025 ***
***                                                                    ***
*** This module implements a logic circuit of simple asynchronous      ***
*** D Flipflop                                                         ***
*************************************************************************/
`timescale 1 ns / 100 ps

module dff (
   input clk,
   input rst_n,     // active low reset
   input d,         // data to the dff
   output reg q     // output must be reg when using in always block
);

   always @(posedge clk or negedge rst_n) begin
      if (!rst_n)
         q <= 1'b0;   // Reset output to 0
      else
         q <= d;      // stores value on the positive edge of the clock
   end

endmodule
```

Appendix C: register8bit.v

```
/*************************************************************************
***                                                                    ***
*** EE 526 L Experiment #4            Muhammad Umar, Spring, 2025      ***
***                                                  ***
*** Experiment #4 Design 8bit register with instances of Mux and DFF   ***
***                                                                    ***
*** Filename: register8bit.v            Created by Muhammad Umar, 03/07/2025
***
***                                                                    ***
*** This module implements a logic circuit of register with instances  ***
*** of Mux and D Flipflops.                                            ***
*************************************************************************/


`timescale 1 ns / 100 ps

module register8bit(
   input clk,
   input rst_n,  // active low reset
   input ena,
   input [7:0] data,
   output [7:0] r
);
   wire [7:0] mux_out;

   // Instantiating Mux and dff module and making multiple instances  and correct port mapping
   MUX2_1 dut_mux0 (.OUT(mux_out[0]), .A(r[0]), .B(data[0]), .SEL(ena));
   dff dut_dff0 (.clk(clk), .rst_n(rst_n), .d(mux_out[0]), .q(r[0]));

   MUX2_1 dut_mux1 (.OUT(mux_out[1]), .A(r[1]), .B(data[1]), .SEL(ena));
   dff dut_dff1 (.clk(clk), .rst_n(rst_n), .d(mux_out[1]), .q(r[1]));

   MUX2_1 dut_mux2 (.OUT(mux_out[2]), .A(r[2]), .B(data[2]), .SEL(ena));
   dff dut_dff2 (.clk(clk), .rst_n(rst_n), .d(mux_out[2]), .q(r[2]));

   MUX2_1 dut_mux3 (.OUT(mux_out[3]), .A(r[3]), .B(data[3]), .SEL(ena));
   dff dut_dff3 (.clk(clk), .rst_n(rst_n), .d(mux_out[3]), .q(r[3]));

   MUX2_1 dut_mux4 (.OUT(mux_out[4]), .A(r[4]), .B(data[4]), .SEL(ena));
   dff dut_dff4 (.clk(clk), .rst_n(rst_n), .d(mux_out[4]), .q(r[4]));

   MUX2_1 dut_mux5 (.OUT(mux_out[5]), .A(r[5]), .B(data[5]), .SEL(ena));
```

```
    dff dut_dff5 (.clk(clk), .rst_n(rst_n), .d(mux_out[5]), .q(r[5]));

    MUX2_1 dut_mux6 (.OUT(mux_out[6]), .A(r[6]), .B(data[6]), .SEL(ena));
    dff dut_dff6 (.clk(clk), .rst_n(rst_n), .d(mux_out[6]), .q(r[6]));

    MUX2_1 dut_mux7 (.OUT(mux_out[7]), .A(r[7]), .B(data[7]), .SEL(ena));
    dff dut_dff7 (.clk(clk), .rst_n(rst_n), .d(mux_out[7]), .q(r[7]));

endmodule
```

Appendix D: tb_register8bit.v

```
/*************************************************************************
***                                                                    ***
*** EE 526 L Experiment #4            Muhammad Umar, Spring, 2025       ***
***                                                                    ***
*** Experiment #4 Design 8bit register with instances of Mux and DFF    ***
***                                                                    ***
*** Filename: tb_register8bit.v     Created by Muhammad Umar, 03/07/2025 ***
***                                                                    ***
*** This module implements a logic circuit of register with instances   ***
*** of Mux and D Flipflops.                                            ***
*************************************************************************/
`timescale 1ns / 100ps

module tb_register8bit;

   reg clk, rst_n, ena;
   reg [7:0] data;
   wire [7:0] r;

   // Instantiate DUT (Device Under Test)
   register8bit dut (
      .clk(clk),
      .rst_n(rst_n),
      .ena(ena),
      .data(data),
      .r(r)
   );

   // Clock Generation: 10 ns period (100 MHz)
   always #5 clk = ~clk;

   initial begin
         $vcdpluson;
      // Initialize Signals
      clk = 0;
      rst_n = 0;    // Active-low reset
      ena = 0;
      data = 8'h00;

         $display("*****************************************************************
***");
```

```
        $display("TEST STARTED - Reset Applied");
        $display("**************************************************************
***");

    $display("\n");
    repeat (2) @(posedge clk);  // Wait for two clock cycles
    rst_n = 1;  // Release reset


    // Test Case 1: Hold previous data when ena = 0
        $display("**************************************************************
***");
        $display("Test Case 1: Hold previous data when ena = 0");
        $display("**************************************************************
***");

        ena = 0;
    data = 8'h8a;  // Set data
    repeat (2) @(posedge clk);
    $display("TEST Case 1: ena = %b, Expected = 00, Observed=%h\n",ena, r);


        // Test Case 2 : Write new data when ena = 1
        $display("**************************************************************
***");
        $display("Test Case 2 : Write new data when ena = 1");
        $display("**************************************************************
***");

    ena = 1;
    repeat (2) @(posedge clk);
        $display("TEST Case 2: ena = %b, Expected = 8a, Observed=%h\n", ena, r);


        // Test Case 3: Check if data holds when ena = 0
        $display("**************************************************************
***");
        $display("Test Case 3: Check if data holds when ena = 0");
        $display("**************************************************************
***");
```

```
        ena = 0;
        data = 8'b1100_0011; // Change input data but should not be stored
        repeat (1) @(posedge clk);
        $display("TEST Case 3: ena = %b, Expected = 8a, Observed=%h\n", ena, r);



            // Test Case 4: Write new data with ena = 1
            $display("**************************************************************
****************************");
            $display("Test Case 4: Write new data with ena = 1, it should reflect on the upcoming
positive edge");
            $display("**************************************************************
****************************");

        ena = 1;
        data = 8'b1111_1111;
        repeat (2) @(posedge clk);
        $display("TEST Case 4: ena=1, Expected= ff, Observed=%h\n", r);



            // Test Case 5: Reset During Operation
            $display("**************************************************************
***");
            $display("Test Case 5: Reset During Operation");
            $display("**************************************************************
***");

        rst_n = 0;
        repeat (2) @(posedge clk);
        $display("TEST Case 5: Reset Applied, Expected=00, Observed=%h\n", r);



            // Test Case 6: clock period is shorter than the critical path delay.
```

```
$display("**********************************************************************
**********");
        $display("Test Case 6: clock period is shorter than the critical path delay which is 14ns.");

$display("**********************************************************************
**********");

        rst_n = 1;  // Release reset
        ena = 1;
        data = 8'haa;  // Load new data
            // Critical path delay is 14 ns, so this should cause timing violations and require 14ns
minumum
        repeat (1) @(posedge clk);
        $display("TEST Case 6: ena=1, Expected= aa, Observed=%h\n", r);

        $display("TEST COMPLETED\n");
        $finish;
    end

    // Monitor Outputs
    initial begin
        $monitor("Time=%0t | rst_n=%b | ena=%b | data=%h | r=%h\n", $time, rst_n, ena, data, r);
    end

endmodule
```

Appendix E: register8bit.log

```
Command: /home/users15/mu832615/526LSP25/LAB4/./simv -l register8bit.log
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version U-2023.03-SP1_Full64; Runtime version U-2023.03-SP1_Full64;
Mar  9 22:35 2025
VCD+ Writer U-2023.03-SP1_Full64 Copyright (c) 1991-2023 by Synopsys Inc.
************************************************************
TEST STARTED - Reset Applied
************************************************************


Time=0 | rst_n=0 | ena=0 | data=00 | r=00

************************************************************
Test Case 1: Hold previous data when ena = 0
```

```
****************************************************************
Time=150 | rst_n=1 | ena=0 | data=8a | r=00

TEST Case 1: ena = 0, Expected = 00, Observed=00

****************************************************************
Test Case 2 : Write new data when ena = 1
****************************************************************
Time=350 | rst_n=1 | ena=1 | data=8a | r=00

Time=450 | rst_n=1 | ena=1 | data=8a | r=8a

TEST Case 2: ena = 1, Expected = 8a, Observed=8a

****************************************************************
Test Case 3: Check if data holds when ena = 0
****************************************************************
Time=550 | rst_n=1 | ena=0 | data=c3 | r=8a

TEST Case 3: ena = 0, Expected = 8a, Observed=8a

****************************************************************************
*************
Test Case 4: Write new data with ena = 1, it should reflect on the upcoming
positive edge
****************************************************************************
*************
Time=650 | rst_n=1 | ena=1 | data=ff | r=8a

Time=750 | rst_n=1 | ena=1 | data=ff | r=ff

TEST Case 4: ena=1, Expected= ff, Observed=ff

****************************************************************
Test Case 5: Reset During Operation
****************************************************************
Time=850 | rst_n=0 | ena=1 | data=ff | r=00

TEST Case 5: Reset Applied, Expected=00, Observed=00

****************************************************************************
******
Test Case 6: clock period is shorter than the critical path delay which is
14ns.
****************************************************************************
******
Time=1050 | rst_n=1 | ena=1 | data=aa | r=ff

TEST Case 6: ena=1, Expected= aa, Observed=ff
```

```
TEST COMPLETED

$finish called from file "tb_register8bit.v", line 120.
$finish at simulation time                 1150
           V C S   S i m u l a t i o n   R e p o r t
Time: 115000 ps
CPU Time:      0.180 seconds;       Data structure size:   0.0Mb
Sun Mar  9 22:35:55 2025
```